MARKDOWN

CHEAT SHEETS

A Complete Reference for CommonMark, GFM & Beyond

Text Formatting · Headings · Lists · Links · Images · Code Blocks · Tables · Callouts · Footnotes · Math · Mermaid · Front Matter · GitHub · Obsidian · Pandoc

22 REFERENCE SHEETS

CONTENTS

BASICS

01 Introduction & Flavors
02 Paragraphs, Line Breaks & Escaping
03 Text Formatting

DOCUMENT STRUCTURE

04 Headings & Anchors
05 Unordered & Ordered Lists
06 Nested, Task & Definition Lists
07 Blockquotes, Callouts & Rules

LINKS, IMAGES & CODE

08 Links
09 Images
10 Inline Code & Code Blocks
11 Syntax Highlighting & Diagrams

TABLES

12 Table Basics & Alignment
13 Complex Tables

EXTENDED SYNTAX

14 Footnotes & Math
15 Front Matter & Attributes
16 Raw HTML & Collapsible Sections

PLATFORMS, TOOLS & PRACTICES

17 GitHub Markdown & READMEs
18 Obsidian & Chat-App Markdown
19 Tooling: Pandoc, Linters & Site Generators
20 Best Practices & Pitfalls

QUICK REFERENCE

21 Quick Reference — Core Syntax
22 Quick Reference — Extended & Flavors

MD

Table of Contents

22 sheets · syntax · examples · notes

BASICS 01 – 03

01 Introduction & Flavors
02 Paragraphs, Line Break…
03 Text Formatting

DOCUMENT STRUCTURE 04 – 07

04 Headings & Anchors
05 Unordered & Ordered …
06 Nested, Task & Definiti…
07 Blockquotes, Callouts …

LINKS, IMAGES & CODE 08 – 11

08 Links
09 Images
10 Inline Code & Code Blo…
11 Syntax Highlighting & …

TABLES 12 – 13

12 Table Basics & Alignm…
13 Complex Tables

EXTENDED SYNTAX 14 – 16

14 Footnotes & Math
15 Front Matter & Attribu…
16 Raw HTML & Collapsibl…

PLATFORMS, TOOLS & PRACTICES 17 – 20

17 GitHub Markdown & R…
18 Obsidian & Chat-App …
19 Tooling: Pandoc, Linter…
20 Best Practices & Pitfalls

QUICK REFERENCE 21 – 22

21 Quick Reference — Cor…
22 Quick Reference — Ext…

Sheet 01

Introduction & Flavors

Gruber 2004 · CommonMark · GFM · Pandoc · MultiMarkdown · Obsidian · editors & preview

WHAT IS MARKDOWN?

Markdown is a lightweight markup language created by **John Gruber** (with Aaron Swartz) in **2004** so people could write HTML as readable plain text. Gruber's description left edge cases undefined, so parsers drifted apart. **CommonMark** (2014) is the strict, unambiguous spec most modern parsers follow, and **GitHub Flavored Markdown (GFM)** is a superset of it that adds tables, task lists, strikethrough and autolinks.

KEY FACTS

- Extensions: `.md` (preferred), `.markdown` ; MIME type `text/markdown` (RFC 7763)
- Gruber's `Markdown.pl` 1.0.1 (2004) was never updated; CommonMark fills the gaps
- Most renderers pass inline HTML through, so HTML is the universal fallback

A MINIMAL DOCUMENT

```
# Project Title                    <!-- <h1> -->

A paragraph with **bold**, *italic* and `code`.

## Install                         <!-- <h2> -->

- Clone the repo                  <!-- <ul><li> -->
- Run the installer

> Tip: [read the docs](https://example.com) first.

```bash
npm install
```
```

CODE EXPLAINED

- → `#` / `##` — heading levels 1 and 2
- → Blank line — separates every block (paragraph, list, heading)
- → `**` `*` `` ` `` — bold, italic, inline code
- → `-` — unordered list item
- → `>` / `[text](url)` — blockquote / link
- → ` ```bash ` — fenced code block with a language

MAJOR FLAVORS

FLAVOR	WHERE	ADDS ON TOP OF COMMONMARK
CommonMark	Reference spec	The baseline: unambiguous core rules
GFM	GitHub, Gitea	Tables · task lists · ~~strike~~ · autolinks
Pandoc	Conversion	Footnotes · math · def lists · citations
MultiMarkdown	Academic	Tables · footnotes · metadata
Markdown Extra	PHP	Footnotes · def lists · {#id}
Obsidian	Notes app	[[wikilinks]] · embeds · callouts
MDX	React docs	JSX components inside Markdown

EDITORS & LIVE PREVIEW

TOOL	HOW TO PREVIEW
VS Code	⌘/Ctrl+Shift+V · side: ⌘/Ctrl+K V
Typora	WYSIWYG: renders as you type
Obsidian	⌘/Ctrl+E toggles reading view
GitHub	Preview tab in any editor box
StackEdit	Browser editor, split preview
CommonMark dingus	Shows the exact HTML output
pandoc	pandoc in.md -o out.html

⚠ COMMON MISTAKES

- Assuming one Markdown — GFM tables render as plain text in strict CommonMark
- Indenting with tabs — 4-column tab stops break list nesting
- Saving as `.txt` — nothing renders it; use `.md`
- No blank lines between blocks — older parsers need them

✓ TIPS & BEST PRACTICES

- Write CommonMark + GFM: the safest common ground
- Check the target renderer before using an extension (PANDOC OBSIDIAN EXT)
- Test tricky input in the CommonMark dingus
- Run `markdownlint` in your editor

QUICK SUMMARY

- `.md` — plain text that a parser renders to HTML
- `CommonMark` — the strict baseline spec
- `GFM` GFM — CommonMark + tables, tasks, strike, autolinks
- `HTML` — inline HTML is the fallback when syntax is missing

Paragraphs, Line Breaks & Escaping

PARAGRAPHS, BREAKS & ESCAPES

A **paragraph** is one or more consecutive lines of text; a **blank line** ends it. A single newline inside a paragraph is a **soft break** and renders as a space, so you can wrap source lines freely. For a **hard break** (
) end the line with a backslash or two spaces. A **backslash** before any ASCII punctuation character makes it literal, and **HTML entities** insert special characters.

RULES

- Up to 3 leading spaces are ignored; 4+ starts an indented code block
- Hard breaks do nothing on the last line of a paragraph
- Escapes and entities are not processed inside code spans or blocks
- HTML comments are hidden from the page but stay in the HTML source

BREAKS, ESCAPES & COMMENTS

```
This line wraps in the source
but renders as one line.          <!-- soft break → space -->

Two trailing spaces··
then a new line.                  <!-- ·· = 2 spaces → <br> -->

A backslash at the end\
then a new line.                  <!-- CommonMark <br> -->

Line one<br>Line two              <!-- raw HTML break -->

\*not italic\*  \# no heading  1986\. no list
&copy; 2026 &mdash; AT&amp;T      <!-- © 2026 — AT&T -->
10 km                        <!-- never wraps -->

<!-- Hidden on the page, kept in the HTML source -->
[//]: # (Not even in the HTML)
```

CODE EXPLAINED

- → Blank line — closes the paragraph and starts a new `<p>`
- → Single newline — soft break, rendered as a space
- → `··` — two trailing spaces = hard break (invisible in source)
- → `\` at line end — visible hard break, preferred
- → `\* \# \.` — escape markup characters
- → `©  ` — HTML entities
- → `[//]: #` — unused link reference, never output

ESCAPABLE CHARACTERS

ESCAPE	STOPS IT FROM…
`\\`	being an escape itself
`` \` ``	starting a code span
`\* \_`	starting emphasis or a list
`\#`	starting a heading
`1\.`	starting an ordered list
`\[ \] \!`	starting a link or image
`\<`	opening an HTML tag

USEFUL ENTITIES

ENTITY	RENDERS
` `	non-breaking space
`&`	&
`< >`	< >
`© ®`	© ®
`— –`	— –
`→ ←`	→ ←
`©`	© (decimal)
`✓`	✓ (hex)

COMMENT STYLES

SYNTAX	BEHAVIOR
`<!-- text -->`	Hidden; can span lines; kept in HTML
`[//]: # (text)`	Link-reference trick; not output
`[comment]: <> (text)`	Same trick, alternate form
`%% text %%`	Hidden in reading view OBSIDIAN

⚠ COMMON MISTAKES

- Relying on trailing spaces — editors strip them and they're invisible in code review
- Expecting a single newline to break the line — it renders as a space
- Escaping inside backticks — `` `\*` `` shows the backslash literally
- Writing `<tag>` in prose — parsed as HTML and vanishes; use backticks or `<`

✓ TIPS & BEST PRACTICES

- Use `\` for hard breaks: visible and survives editors
- Write one sentence per line — soft breaks keep git diffs small
- Use ` ` to keep units and numbers together
- Don't put secrets in `<!-- -->` — they ship in the HTML

QUICK SUMMARY

- `blank line` — ends a paragraph
- `newline` — soft break, renders as a space
- `\` / `··` — hard line break
- `\char` — literal punctuation
- `&name;` — HTML entity

Text Formatting

italic · **bold** · ***both*** · ~~strike~~ · ==highlight== · H~2~O / x^2^ · intraword rules

EMPHASIS & INLINE STYLES

Emphasis wraps text in * or _ delimiters: one for **italic** (), two for **bold** (), three for both. Delimiters must touch the text they wrap. **Strikethrough** is a GFM extension; highlight, subscript and superscript exist only in some flavors, so inline **HTML tags** are the portable fallback.

DELIMITER RULES

- `*` works inside words: `un*frigging*believable`
- `_` does not: `snake_case_name` stays literal
- `* text *` with inner spaces is not emphasis
- Emphasis cannot span a blank line (paragraph boundary)
- No underline syntax — use `<ins>` or `<u>`

FORMATTING EXAMPLES

```
*italic*   _italic_              <!-- <em> -->
**bold**   __bold__              <!-- <strong> -->
***bold italic***                <!-- <em><strong> -->
**bold with *italic* inside**
~~strikethrough~~                <!-- GFM <del> -->
==highlighted==                  <!-- Obsidian <mark> -->
H~2~O   x^2^                     <!-- Pandoc sub / sup -->
H<sub>2</sub>O  x<sup>2</sup>    <!-- HTML: portable -->
<mark>marked</mark> <ins>underlined</ins>
<kbd>Ctrl</kbd>+<kbd>C</kbd>     <!-- keyboard keys -->
un*frigging*believable           <!-- intraword: italic -->
snake_case_name                  <!-- stays literal -->
2 \* 3 = 6                       <!-- escaped asterisk -->
```

CODE EXPLAINED

- → `*x*` / `_x_` — italic
- → `**x**` / `__x__` — bold
- → `***x***` — bold + italic; nesting also works
- → `~~x~~` — strikethrough GFM
- → `==x==` — highlight OBSIDIAN
- → `~x~` `^x^` — sub / superscript PANDOC
- → `<sub>` `<kbd>` — HTML fallbacks

SYNTAX → RESULT

SYNTAX	RENDERS
`*text*`	*text*
`**text**`	**text**
`***text***`	***text***
`~~text~~`	~~text~~ GFM
`==text==`	text OBSIDIAN
`H~2~O`	H_2O PANDOC
`x^2^`	x^2 PANDOC

HTML FALLBACKS

TAG	USE
`<sub> <sup>`	Sub / superscript
`<mark>`	Highlight
`<ins> <u>`	Underline / inserted
`<del> <s>`	Strikethrough
`<kbd>`	Keyboard key
`<small>`	Fine print
`<abbr title="">`	Abbreviation tooltip

FLAVOR SUPPORT

FEATURE	CM	GFM	PANDOC	OBS
`* ** ***`	✔	✔	✔	✔
`~~strike~~`	✖	✔	✔	✔
`==highlight==`	✖	✖	✖	✔
`~sub~`	✖	✖	✔	✖
`^sup^`	✖	✖	✔	✖
`<sub> <kbd>`	✔	✔	✔	✔
`<u> <ins>`	✔	✔	✔	✔

⚠ COMMON MISTAKES

- `** bold **` — spaces inside delimiters print the asterisks literally
- `H~2~O` on GitHub — single tildes are strikethrough there; use `<sub>`
- `__init__` in prose renders as bold **init** — wrap code in backticks
- Mixing delimiters `*text_` — openers and closers must match

✓ TIPS & BEST PRACTICES

- Pick `*` for italic and `**` for bold and stay consistent
- Use `<kbd>` for shortcuts: `Ctrl+S`
- Bold sparingly — for UI labels and warnings, not whole sentences
- Escape math and globs: `2 \* 3`, `\*.md`

QUICK SUMMARY

- `*x*` — italic
- `**x**` — bold
- `~~x~~` — strikethrough (GFM)
- `<sub> <sup> <mark>` — portable extras
- `\*` — literal asterisk

Headings & Anchors

to ###### · setext === / --- · auto IDs · slug rules · {#custom-id} · [link](#heading)

HEADINGS & ANCHORS

ATX headings start with 1–6 # characters followed by a space; the count sets the level (<h1>–<h6>). **Setext headings** underline text with = (level 1) or – (level 2). Most renderers also give every heading an **id** (a slug of its text), so you can link to it with `[text](#slug)`.

RULES

- A space after `#` is required: `#Title` is a paragraph
- Closing `#`s are optional and ignored: `## Title ##`
- Use one `#` H1 per document and don't skip levels

HEADING SYNTAX

```
# Heading 1                       <!-- <h1 id="heading-1"> -->
## Heading 2
### Heading 3 ###                 <!-- closing #s ignored -->
#### Heading 4                    <!-- … down to ###### (h6) -->
#Not a heading                    <!-- no space → paragraph -->

Setext Heading 1
================

Setext Heading 2
----------------

## Installation {#setup}          <!-- Pandoc / Extra / MkDocs -->

Jump to [Heading 2](#heading-2) or [Setup](#setup).
See [Usage](guide.md#usage) in another file.
```

CODE EXPLAINED

- → `#` ×1–6 + space — heading level 1–6
- → `===` / `---` under text — setext h1 / h2 only
- → Auto id — lowercase slug of the heading text
- → `{#setup}` — custom id PANDOC EXT
- → `(#id)` / `(file.md#id)` — link to a heading here / in another file

GITHUB SLUG RULES

HEADING	ANCHOR
`## Getting Started`	`#getting-started`
`## API v2.0`	`#api-v20`
`## C++ & Rust`	`#c--rust`
`## What's New?`	`#whats-new`
``## `config` file``	`#config-file`
`## Setup ×2`	`#setup, #setup-1`

HOW SLUGS ARE BUILT

STEP	EFFECT
1. lowercase	API → api
2. strip punctuation	Drops punctuation except – _
3. spaces → -	Each space becomes one hyphen
4. de-duplicate	Repeats get –1, –2 …
Markup ignored	**bold** / `code` → text only

CUSTOM IDS BY FLAVOR

FLAVOR	SYNTAX
Pandoc / Extra	`## Title {#id}`
MkDocs	`## Title { #id }` (attr_list)
GitHub	None — add
Obsidian	Link by text: `[[#Heading]]`
Any HTML	<h2 id="id">Title</h2>

⚠ COMMON MISTAKES

- `#Heading` without a space — renders as a paragraph in CommonMark
- Skipping levels (h1 → h3) — breaks the outline and screen-reader navigation
- `---` right under a paragraph line makes it an h2; add a blank line for a rule
- Linking `#Getting Started` — anchors are lowercase and hyphenated

✓ TIPS & BEST PRACTICES

- Prefer ATX: works for all six levels and is easy to grep
- Generate a TOC from headings (`pandoc --toc` , VS Code "Markdown All in One")
- Keep headings short and stable — renaming one breaks every link to it
- Check the real anchor by hovering the heading's link icon on GitHub

QUICK SUMMARY

- `# … ######` — heading levels 1–6
- `===` / `---` — setext h1 / h2
- `#slug` — auto id: lowercase, hyphens
- `{#id}` — custom id (Pandoc, Extra, MkDocs)
- `[text](#id)` — link to a heading

Sheet 05

Unordered & Ordered Lists

- * + · 1. / 1) · start number · all-1. numbering · tight vs loose · escaping 1986\.

UNORDERED & ORDERED LISTS

An **unordered** item starts with –, ∗ or + plus a space. An **ordered** item starts with a number followed by `.` or `)`. Only the **first number** matters: it sets the start value, and later numbers are ignored. A list is **tight** when items have no blank lines between them and **loose** when they do; loose items get wrapped in tags, which adds vertical space.

RULES

- Changing the marker (`-` → `*`) or delimiter (`.` → `)`) starts a new list
- An ordered list can interrupt a paragraph only if it starts at `1`
- List numbers can be up to 9 digits; `0.` is a valid start
- Escape the period to stop accidental lists: `1986\.`

LIST SYNTAX

```
- Apples                    <!-- tight <ul> -->
- Oranges

* Star marker               <!-- new list: marker changed -->
+ Plus marker               <!-- and another new list -->

1. All ones                 <!-- <ol>: renders 1, 2, 3 -->
1. Easy to reorder
1. Clean diffs

7. Starts at seven          <!-- <ol start="7"> -->
8. Eight

1) Paren delimiter          <!-- also valid -->

- Loose item one            <!-- loose: <li><p>…</p></li> -->

- Loose item two

1986\. A great year.        <!-- escaped: not a list -->
```

CODE EXPLAINED

- → `- * +` — unordered markers, need a space after
- → `1.` / `1)` — ordered markers
- → Start number — first item sets `start="N"`
- → All `1.` — renderer numbers them in order
- → Blank lines between items — loose list, more spacing
- → `\.` — escaped period, plain paragraph

MARKERS

SOURCE	RESULT
`- item`	bullet
`* item`	bullet
`+ item`	bullet
`1. item`	numbered
`1) item`	numbered
`-item`	paragraph (no space)

NUMBERING

SOURCE NUMBERS	RENDERS
`1. 2. 3.`	1, 2, 3
`1. 1. 1.`	1, 2, 3
`3. 1. 9.`	3, 4, 5
`0. 0. 0.`	0, 1, 2
`1. then 1)`	two separate lists

TIGHT VS LOOSE

LAYOUT	HTML
`no blank lines`	text
`blank lines`	text
`one blank anywhere`	whole list is loose
`<!-- --> between`	splits two lists

⚠ COMMON MISTAKES

- `-item` with no space — renders as a paragraph, not a list
- One stray blank line between items makes the whole list loose (extra spacing)
- Switching from `-` to `*` mid-list — silently starts a new list
- A sentence starting `2020. Great year` — becomes list item 2020

✓ TIPS & BEST PRACTICES

- Use `-` everywhere (markdownlint MD004 enforces one style)
- Use all `1.` for long, reordered lists; real numbers when text cites step numbers
- Leave a blank line before and after every list
- Put `<!-- -->` between two lists that should stay separate

QUICK SUMMARY

- `- item` — unordered list
- `1. item` — ordered list
- `N.` first — sets the start number
- `blank lines` — loose list with `<p>`
- `1986\.` — escape to avoid a list

Sheet 06

Nested, Task & Definition Lists

indent width · multi-paragraph items · code in lists · - [ ] · - [x] · Term + : definition

NESTING, TASKS & DEFINITIONS

To **nest** anything inside a list item (a sub-list, a second paragraph, a code block), indent it to the item's **content column**: the first character after the marker and its space. That's 2 spaces under – and 3 under `1.` . **Task lists** (GFM) add `[ ]` or `[x]` after the marker. **Definition lists** pair a term with `:` definitions and only work in Pandoc, PHP Markdown Extra and MkDocs.

RULES

- 4 spaces is safe under both `-` and `1.` , and is *required* by Python-Markdown / MkDocs
- Indenting past the content column by 4+ spaces makes an indented code block
- Lazy continuation: an unindented line still continues the item's paragraph
- GitHub task boxes are clickable in issues and PRs and edit the source

NESTED, TASK & DEFINITION LISTS

```
- Fruit
  - Apple                        <!-- 2 spaces: under "- " -->
    - Granny Smith               <!-- 3rd level -->

1. Install
   1. Download                   <!-- 3 spaces: under "1. " -->
   2. Run the installer

   A second paragraph in item 1. <!-- same indent + blank line -->

   ```bash
   npm install
   ```
2. Configure

- [x] Write the outline   <!-- GFM task list -->
- [ ] Build the sheets
  - [ ] Sheet 06   <!-- nested task -->

Markdown   <!-- Pandoc / Extra / MkDocs -->
:   A lightweight markup language.
:   Created by John Gruber.
```

CODE EXPLAINED

- → Child indent = width of parent marker + 1 space
- → Blank line + indent — extra paragraph inside the item
- → Fence indented to the content column — stays in the list
- → `[ ]` / `[x]` — unchecked / checked task GFM
- → `Term` + `: def` — definition list PANDOC EXT

CONTENT COLUMN

PARENT	CHILD INDENT
`- item`	2 spaces
`1. item`	3 spaces
`10. item`	4 spaces
`- item`	4 spaces (3 after marker)
MkDocs	always 4 spaces

TASK LISTS

SYNTAX	RESULT
`- [ ] todo`	☐ unchecked
`- [x] done`	☑ checked
`- [X] done`	☑ checked (either case)
`* [ ]` / `1. [ ]`	any list marker works
`- []` todo	✖ no space → plain text

DEFINITION LIST SUPPORT

FLAVOR	SUPPORT
Pandoc	✔ `Term` / `: def`
PHP Extra	✔ `: def`
MkDocs	✔ `def_list` extension
GitHub	✖ use <dl><dt><dd>
Obsidian	✖ use HTML or bold terms

⚠ COMMON MISTAKES

- Indenting a child by 1 space — not nested, it becomes a sibling item
- Code fence at column 0 inside an item — ends the list early
- 2-space nesting in MkDocs — Python-Markdown needs 4, so lists flatten
- `[]` without a space — not a task box

✓ TIPS & BEST PRACTICES

- Set your editor to insert spaces (not tabs) for list indents
- Use task lists in PR descriptions and READMEs for checklists
- On GitHub, write definition lists in HTML `<dl>`
- Preview deeply nested lists — renderers disagree most here

QUICK SUMMARY

- `content column` — indent children to it
- `- child` — 2 spaces under `-`
- `1. child` — 3 spaces under `1.`
- `- [ ]` / `- [x]` — task list (GFM)
- `: def` — definition (Pandoc, Extra, MkDocs)

Sheet 07

Blockquotes, Callouts & Rules

> · nested >> · > [!NOTE] · [!TIP] · [!WARNING] · Obsidian callouts · --- *** · page breaks

QUOTES, CALLOUTS & RULES

A **blockquote** prefixes lines with > and can hold any block: paragraphs, lists, headings, code. Nest with >>. **GitHub alerts** turn a blockquote into a colored callout when its first line is `[!NOTE]`, `[!TIP]`, `[!IMPORTANT]`, `[!WARNING]` or `[!CAUTION]`. A **horizontal rule** is 3 or more –, ∗ or _ on a line by itself.

QUOTES, ALERTS & RULES

```
> Quoted paragraph,
> still the same quote.
>
> Second paragraph in the quote.
> - lists, **bold**, `code` work too
>
>> Nested quote.

> [!NOTE]                        <!-- GitHub alert -->
> Useful information.

> [!tip]- Click to expand        <!-- Obsidian: folded callout -->
> Hidden until opened.

---                              <!-- horizontal rule -->
```

RULES

- Lazy continuation: a line without `>` still continues the quoted paragraph
- A blank line ends the quote; a lone `>` keeps it going
- Keep the alert marker alone on its line; content goes on the next lines

CODE EXPLAINED

- → `>` — quote; any block can go inside
- → `>` alone — blank line that stays inside the quote
- → `>>` — nested quote
- → `[!NOTE]` — GitHub alert GFM
- → `[!tip]-` — collapsed Obsidian callout OBSIDIAN
- → `---` — `<hr>` (needs a blank line above)

GITHUB ALERTS

MARKER	COLOR	USE FOR
`[!NOTE]`	Blue	Useful info to skim past
`[!TIP]`	Green	Helpful advice
`[!IMPORTANT]`	Purple	Key info for success
`[!WARNING]`	Yellow	Urgent; needs attention
`[!CAUTION]`	Red	Risky or harmful outcomes

OTHER CALLOUT SYNTAXES

SYNTAX	MEANING
`> [!info] Title`	Obsidian, custom title OBSIDIAN
`> [!faq]+`	Foldable, open by default OBSIDIAN
`> [!faq]-`	Foldable, closed OBSIDIAN
`!!! note "Title"`	MkDocs admonition EXT
`??? note`	MkDocs collapsible EXT

RULES & BREAKS

SYNTAX	RESULT
`--- *** ___`	<hr>
`- - - ------`	<hr> (spaces, any length)
`--`	✖ too short: text
`Text ---`	✖ setext <h2>
`\newpage`	PDF page break PANDOC

⚠ COMMON MISTAKES

- Two quotes with only a blank line between them — some renderers merge them; add text or `<!-- -->`
- `---` right under a paragraph — becomes an h2, not a rule
- Alerts outside GitHub — other renderers show the literal `[!NOTE]`
- Text on the `[!NOTE]` line — GitHub then renders a plain quote

✓ TIPS & BEST PRACTICES

- Use alerts sparingly — GitHub's docs suggest one or two per page
- Use a lone `>` to keep multi-paragraph quotes together
- Use `---` for rules consistently (markdownlint MD035)
- Prefer headings over rules to structure a document

QUICK SUMMARY

- `>` — blockquote; `>>` nests
- `> [!NOTE]` — GitHub alert (5 types)
- `> [!type]-` — Obsidian callout, foldable
- `---` — horizontal rule
- `\newpage` — Pandoc PDF page break

Links

[text](url "title") · [text][id] · [id]: url · <https://…> · GFM bare URLs · mailto: · file.md#section

LINKS

An **inline link** puts the URL right after the text: `[text](url "title")`. A **reference link** uses a label, `[text][label]`, and defines the URL once elsewhere with `[label]: url`; definitions never render. **Autolinks** wrap a URL in angle brackets, and GFM also links bare `https://` and `www.` URLs. **Relative paths** link to other files in the same repo or site.

RULES

- No space between `]` and `(` — `[a] (b)` is not a link
- Reference labels are case-insensitive; define them anywhere in the file
- Titles can use `"…"`, `'…'` or `(…)`
- URLs with spaces: wrap in `<…>` or encode as `%20`

LINK SYNTAX

```
[Markdown Guide](https://www.markdownguide.org)
[Guide](https://example.com "Hover title")      <!-- title -->
[Install notes](docs/install.md)                <!-- relative file -->
[Usage](docs/install.md#usage)                  <!-- file + heading -->
[Jump to FAQ](#faq)                             <!-- same page -->
[Spec](<docs/my spec.md>)                       <!-- spaces in path -->
[Email us](mailto:team@example.com)

Read the [spec][cm] and the [GFM spec][].       <!-- full / collapsed -->
More in [cm].                                   <!-- shortcut -->

[cm]: https://spec.commonmark.org "CommonMark"
[GFM spec]: https://github.github.com/gfm/

<https://example.com>                           <!-- autolink -->
Visit www.example.com today.                    <!-- GFM bare URL -->
```

CODE EXPLAINED

- → `[text](url)` — inline link; `"title"` = tooltip
- → `docs/file.md` — path relative to this file
- → `#faq` — heading anchor (see Sheet 04)
- → `[text][label]` + `[label]: url` — reference link
- → `[label][]` / `[label]` — collapsed / shortcut forms
- → `<url>` — autolink; bare URLs need GFM

LINK FORMS

SYNTAX	TYPE
`[t](url)`	Inline
`[t](url "Title")`	Inline with title
`[t][label]`	Full reference
`[label][]`	Collapsed reference
`[label]`	Shortcut reference
`<https://x.com>`	Autolink
`<me@x.com>`	Email autolink
`www.x.com`	Bare URL GFM

PATH TARGETS

PATH	POINTS TO
`file.md`	Same folder
`docs/a.md`	Subfolder
`../README.md`	Parent folder
`/docs/a.md`	Repo / site root
`#faq`	Heading on this page
`a.md#faq`	Heading in another file
`mailto:a@b.com`	Opens email client

SPECIAL CASES

PROBLEM	FIX
`Spaces in URL`	`<my file.md>` or `%20`
`Parens in URL`	Balanced OK; else `%28` `%29`
`] in link text`	Escape it: `\]`
`Open in new tab`	No syntax; use `<a target>`
`Image as link`	`[![alt](img)](url)`
`Bold link`	`**[text](url)**`

⚠ COMMON MISTAKES

- `[text] (url)` — a space breaks the link; both parts print literally
- Using an undefined label — `[text][x]` renders as plain brackets
- Absolute `github.com/…/blob/main/` links to repo files — break on forks and branches
- "Click here" link text — tells readers and screen readers nothing

✓ TIPS & BEST PRACTICES

- Write link text that makes sense alone: `[install guide](…)`
- Use relative links for files in the same repo
- Collect reference definitions at the end of the section or file
- Reference links keep long URLs out of paragraphs and diffs

QUICK SUMMARY

- `[t](url)` — inline link
- `[t][id]` + `[id]: url` — reference link
- `<url>` — autolink
- `(#id)` — heading anchor
- `(../a.md)` — relative file link

Images

IMAGES

An image is a link with a leading `!`: `![alt text](path "title")`. The **alt text** is read by screen readers and shown if the file fails to load. Reference syntax works too: `![alt][label]`. CommonMark and GFM have **no sizing syntax**, so use an HTML `<img>` tag to set width or alignment. Wrapping an image in a link makes it clickable.

GOOD TO KNOW

- Any format the browser shows works: PNG, JPG, GIF, SVG, WebP
- Relative paths resolve from the Markdown file's folder
- Drag an image into a GitHub issue or PR to upload it
- Use `alt=""` only for purely decorative images

IMAGE SYNTAX

```
![Project logo](images/logo.png)                    <!-- relative -->
![Logo](images/logo.png "Shown on hover")           <!-- title -->
![Build pipeline][pipeline]                         <!-- reference -->

[pipeline]: assets/pipeline.svg

[![CI](https://img.shields.io/badge/CI-passing-green)](https://ci.example.com)

<img src="images/logo.png" alt="Logo" width="200">  <!-- sized -->
<p align="center"><img src="shot.png" alt="App" width="60%"></p>

![Photo](photo.jpg){width=50%}                      <!-- Pandoc -->
![[diagram.png|300]]                                <!-- Obsidian -->

<picture>                                           <!-- dark mode -->
  <source media="(prefers-color-scheme: dark)" srcset="logo-dark.png">
  <img alt="Logo" src="logo-light.png">
</picture>
```

CODE EXPLAINED

- → `![alt](src)` — basic image
- → `![alt][label]` — reference image
- → `[![…](badge)](url)` — clickable image / badge
- → `<img width>` — the portable way to size
- → `{width=50%}` PANDOC · `|300` OBSIDIAN
- → `<picture>` — light / dark variants on GitHub

RESIZING BY FLAVOR

FLAVOR	ADD AFTER THE IMAGE
CommonMark / GFM	Use `<img width="300">`
Pandoc	`{width=50%}`
MkDocs	`{ width="300" }`
kramdown	`{: width="300"}`
Obsidian	`![[a.png|300]]`

WRITING ALT TEXT

IMAGE	ALT TEXT
Screenshot	What it shows: "Settings page, Save button"
Diagram	The takeaway: "Request flows A → B → C"
Logo	"Acme logo"
Badge	"CI passing"
Decorative	`alt=""` (HTML)

COMMON PATTERNS

GOAL	HOW
Center	`<p align="center">`
Side by side	Two `<img>` on one line
Caption	Italic line below, or Pandoc figure
Dark mode	`<picture>` + `<source>`

⚠ COMMON MISTAKES

- Empty or filename alt text (`![](a.png)` , `![IMG_2041]`) — useless to screen readers
- Local absolute paths like `C:\Users\…` — break for everyone else
- Spaces in file names — wrap in `<…>` or rename with hyphens
- Using `{width=…}` on GitHub — prints literally; use `<img>`

✓ TIPS & BEST PRACTICES

- Keep images in an `images/` or `assets/` folder next to the docs
- Use SVG for diagrams and compress PNG/JPG screenshots
- Describe the point of the image, not its appearance
- Set only `width` so the image keeps its aspect ratio

QUICK SUMMARY

- `![alt](src)` — image
- `[![alt](src)](url)` — linked image
- `<img width>` — portable sizing
- `alt` — required for accessibility
- `<picture>` — dark-mode variants

Inline Code & Code Blocks

INLINE CODE & CODE BLOCKS

Inline code wraps text in backticks; everything inside is literal, so escapes and Markdown don't apply. To include a backtick, use a longer backtick run as the delimiter. **Fenced code blocks** start and end with 3+ backticks or tildes; the word after the opening fence (the **info string**) names the language. An older form, the **indented code block**, is any text indented 4 spaces.

RULES

- The closing fence must use the same character and be at least as long as the opening one
- An unclosed fence runs to the end of the document
- One leading + trailing space inside `` `x` `` is stripped
- An indented code block cannot interrupt a paragraph

CODE SYNTAX

````
Run `npm install` before `npm start`.
`` Use `backticks` inside ``
Press <kbd>Ctrl</kbd>+<kbd>C</kbd> to stop.

```python
print("Hello, Markdown")
```

~~~
Tildes work as fences too.
~~~

````markdown
```js
console.log("a fence inside a fence");
```
````

    Indented 4 spaces = code block.
````

CODE EXPLAINED

- → `` `code` `` — inline `<code>`
- → `` `` `x` `` `` — longer delimiter lets backticks inside
- → ` ```python ` — fenced block; `python` = info string
- → `~~~` — tilde fence, same behavior
- → ` ```` ` — 4-backtick fence shows a 3-backtick fence
- → 4-space indent — indented code block (no language)

INLINE CODE

SOURCE	RENDERS
`` `code` ``	code
`` ``a`b`` ``	a`b
`` `` `x` `` ``	`x`
`` `\*` ``	* (no escapes)
`` `<b>` ``	<b> (no HTML)

FENCE RULES

RULE	DETAIL
` ``` ` or `~~~`	3+ of one character
Closing fence	Same char, ≥ length
Info string	First word = language
No closing fence	Runs to end of file

FENCED VS INDENTED

FEATURE	FENCED	INDENTED
Language	✔	✖
Breaks a paragraph	✔	✖
Inside lists	Easy	+4 more spaces
Recommended	✔	✖

⚠ COMMON MISTAKES

- An unclosed fence — the rest of the document becomes code
- Typing `'''` (quotes) instead of ` ``` ` (backticks)
- Indenting a paragraph 4 spaces for looks — it turns into a code block
- Trying `\` ` inside inline code — use a longer delimiter instead

✓ TIPS & BEST PRACTICES

- Always add a language after the fence for highlighting
- Show fences in docs with `~~~` or 4 backticks around them
- Use `<kbd>`, not backticks, for key presses
- Keep code lines short (~80 chars) so they don't scroll

QUICK SUMMARY

- `` `x` `` — inline code
- ` ```lang ` — fenced block with language
- `~~~` — alternate fence
- ` ```` ` — wraps a ``` example
- `4 spaces` — indented code block

Sheet 11

Syntax Highlighting & Diagrams

language IDs · diff · console · line highlight {2-4} · ```mermaid · flowchart · sequenceDiagram

HIGHLIGHTING & DIAGRAMS

The language in a fence's info string picks the **syntax highlighter** grammar (GitHub uses Linguist; other tools use Prism, highlight.js, Shiki or Pygments). Special languages like `diff` and `console` color added lines or prompts. **Line highlighting, titles and line numbers** are site-generator features with tool-specific syntax. A `mermaid` fence renders a **diagram** from text on GitHub, GitLab, Obsidian and many doc tools.

GOOD TO KNOW

- Unknown language → plain, unhighlighted text
- Aliases usually work: `js`, `sh`, `yml`, `py`
- GitHub also renders `geojson`, `topojson` (maps) and `stl` (3D) fences
- GitHub ignores titles and line highlights in the info string

HIGHLIGHTING & MERMAID

````
```diff
- const port = 3000;
+ const port = 8080;
```

```console
$ npm test
All 42 tests passed
```

```python title="app.py" hl_lines="2"
def main():
    print("highlighted line")
```

```mermaid
flowchart LR
    A[Write .md] --> B{Lint passes?}
    B -->|yes| C[Publish]
    B -->|no| A
```
````

CODE EXPLAINED

- → `diff` — `-` lines red, `+` lines green
- → `console` — prompt + output (`shell-session` on GitHub)
- → `title=` / `hl_lines=` — MkDocs Material EXT
- → `mermaid` — rendered diagram, not code
- → `A[..]` box · `B{..}` decision · `-->` arrow · `|yes|` label

COMMON LANGUAGE IDS

ID	USE FOR
`bash sh shell`	Shell scripts
`console`	Prompt + output
`diff`	Patches, before/after
`yaml yml`	Config, front matter
`js ts jsx tsx`	JavaScript family
`text plaintext`	No highlighting

EXTRAS BY TOOL

INFO STRING	EFFECT · TOOL
`{2,4-6}`	Highlight lines · Docusaurus, VitePress
`hl_lines="2 4"`	Highlight lines · MkDocs
`title="app.py"`	File title · Docusaurus, MkDocs
`linenums="1"`	Line numbers · MkDocs
`:line-numbers`	Line numbers · VitePress

MERMAID DIAGRAM TYPES

KEYWORD	DRAWS
`flowchart LR`	Flowchart (LR, TD …)
`sequenceDiagram`	Messages over time
`classDiagram`	Classes and relations
`stateDiagram-v2`	State machine
`erDiagram`	Database entities
`gantt`	Project timeline

⚠ COMMON MISTAKES

- Misspelled language (```` ```pyhton ````) — silently renders unhighlighted
- Using `bash` for terminal sessions — `$` prompts get highlighted as code
- Expecting Mermaid in every renderer — Pandoc and plain CommonMark show raw text
- Copying Docusaurus `{2}` syntax into MkDocs, or the reverse

✓ TIPS & BEST PRACTICES

- Use `console` for commands with output, `bash` for scripts
- Use `diff` to show before/after changes in docs
- Test diagrams in the Mermaid Live Editor (mermaid.live)
- Keep diagrams small; split big ones into several

QUICK SUMMARY

- ```` ```lang ```` — choose highlighter grammar
- `diff` / `console` — special highlighting
- `{2}` / `hl_lines` — tool-specific extras
- ```` ```mermaid ```` — text-to-diagram

Table Basics & Alignment

PIPE TABLES

A **pipe table** (a GFM extension, not core CommonMark) has a **header row**, a **delimiter row** of dashes, then body rows, with | between cells. Colons in the delimiter row set each column's **alignment**. Cells can hold inline Markdown (bold, code, links, images) but not block content like lists or code blocks. Columns don't need to line up in the source.

RULES

- Header and delimiter rows must have the same number of cells
- Outer pipes are optional but make tables easier to read
- Short body rows get empty cells; extra cells are dropped
- Put a blank line before the table so it isn't read as a paragraph
- Escape a pipe inside a cell as `\|`, even inside backticks

TABLE SYNTAX

```
| Command  | Description            | Default |
|:---------|:----------------------:|--------:|
| `init`   | Create a project       |     n/a |
| `build`  | Compile **all** files  |      on |
| `serve`  | Start the dev server   |    3000 |
| `a \| b` | Escaped pipe in a cell |       – |
| `test`   | Line one<br>Line two   |         |

Minimal form (no outer pipes, default alignment):

Name | Age
--- | ---
Ada | 36
```

CODE EXPLAINED

- → Row 1 — header cells become `<th>`
- → `:---` left · `:---:` center · `---:` right
- → `**bold**`, `` `code` ``, links — inline Markdown works
- → `\|` — literal pipe inside a cell
- → `<br>` — line break inside a cell
- → Outer pipes and padding are optional

ALIGNMENT

DELIMITER	ALIGNS
`:---`	Left
`:---:`	Center
`---:`	Right
`---`	Default (left)

INSIDE A CELL

CONTENT	WORKS?
`**bold** `code``	✔
`[link](url) ![img](src)`	✔
`  |`	✔
`- list / 1. list`	✖ use • or HTML
``` ``` code block ```	✖ use HTML table
`Two paragraphs`	✖ use

PIPE TABLE SUPPORT

FLAVOR	SUPPORT
`CommonMark`	✖ core has no tables
`GFM / GitHub`	✔
`Pandoc`	✔ + grid, multiline
`Obsidian`	✔
`MkDocs`	✔ (tables ext, on by default)
`MultiMarkdown`	✔ + colspan ||

⚠ COMMON MISTAKES

- No blank line before the table — the rows print as one paragraph
- Delimiter row with a different cell count than the header — not a table
- Unescaped `|` in a cell (even in `` `code` ``) — splits the column
- Putting lists or code blocks in cells — use an HTML table (Sheet 13)

✓ TIPS & BEST PRACTICES

- Let a formatter align columns: Prettier or VS Code "Markdown All in One"
- Keep tables narrow; split wide ones or move detail below
- Right-align number columns with `---:`
- Use tablesgenerator.com to paste from a spreadsheet

QUICK SUMMARY

- `| a | b |` — table row
- `|---|---|` — delimiter row (required)
- `:---:` — center; `---:` right
- `\|` — literal pipe
- `<br>` — line break in a cell

Complex Tables

WHEN PIPE TABLES AREN'T ENOUGH

Pipe tables can't **merge cells**, hold **block content** (lists, code blocks, paragraphs) or set **column widths**. The portable answer is an **HTML <table>**, which GitHub and most renderers accept, with `colspan` and `rowspan` for merged cells. Pandoc adds **grid tables** drawn with +, – and | that allow block content, and MultiMarkdown supports column spans.

GOOD TO KNOW

- GitHub keeps `colspan`, `rowspan` and `align` but strips `style`
- Markdown inside an HTML cell needs a blank line before and after it
- Grid table borders must line up exactly, character for character
- Pandoc caption: a `Table: …` or `: …` line next to the table

HTML & GRID TABLES

```
<table>
  <tr><th rowspan="2">Plan</th><th colspan="2">Limits</th></tr>
  <tr><th>Users</th><th>Storage</th></tr>
  <tr><td>Free</td><td>1</td><td>5 GB</td></tr>
  <tr><td>Pro</td><td colspan="2">

**Unlimited** – see [pricing](pricing.md)

  </td></tr>
</table>

+-------+-------------------+
| Plan  | Features          |
+=======+===================+
| Free  | - 1 user          |
|       | - 5 GB storage    |
+-------+-------------------+
```

CODE EXPLAINED

→ `rowspan="2"` — cell spans two rows

→ `colspan="2"` — cell spans two columns

→ Blank lines around `**Unlimited**` — Markdown parsed inside the cell

→ `+===+` — grid table header separator PANDOC

→ Grid cells can hold lists and code blocks PANDOC

PICK THE RIGHT TOOL

NEED	USE
Merged cells	HTML `colspan` / `rowspan`
Lists in cells	HTML, or Pandoc grid table
Code in cells	HTML `<pre>` or grid table
Column spans	MultiMarkdown `| a ||`
Caption	Pandoc `Table:` / HTML `<caption>`

PANDOC TABLE TYPES

EXTENSION	LOOKS LIKE
pipe_tables	GFM-style pipes
simple_tables	Columns under dashes, no pipes
multiline_tables	Rows can wrap; widths from dashes
grid_tables	+---+ borders; block content

GENERATORS & HELPERS

TOOL	DOES
tablesgenerator.com	Visual editor → MD / HTML
Prettier	Re-aligns pipe tables
Markdown All in One	VS Code table formatting
pandoc -f csv -t gfm	CSV → pipe table
Emacs table.el / org	Draws grid tables

⚠ COMMON MISTAKES

- No blank lines around Markdown in `<td>` — `**text**` prints literally
- Relying on `style="…"` widths on GitHub — they're stripped
- Off-by-one border in a grid table — the whole table falls back to text
- Indenting HTML table rows 4+ spaces after a blank line — becomes a code block

✓ TIPS & BEST PRACTICES

- Try a pipe table first; reach for HTML only when you need it
- Keep HTML tables compact: one `<tr>` per line
- For huge tables, link to a CSV instead of inlining it
- Use a formatter to draw grid tables; never align by hand

QUICK SUMMARY

- `<table>` — portable complex tables
- `colspan` / `rowspan` — merged cells
- blank lines — Markdown inside HTML cells
- `+---+` — Pandoc grid table
- `Table: caption` — Pandoc caption

Footnotes & Math

FOOTNOTES & MATH

A **footnote** has a marker in the text, `[^label]`, and a definition anywhere in the file, `[^label]: note text`. Notes are collected at the bottom, **numbered in order of first use**, with links back to the text. **Math** uses LaTeX between dollar signs: `$…$` inline, `$$…$$` on its own lines for display math. GitHub renders math with MathJax; most other tools use KaTeX.

GOOD TO KNOW

- Labels can be words (`[^gruber]`); output is still numbered
- Indent 4 spaces to add paragraphs to a footnote
- Opening `$` must not be followed by a space
- Escape currency as `\$` where math is enabled
- Neither footnotes nor math are in core CommonMark

FOOTNOTES & MATH

```
Markdown was released in 2004.[^1]
Gruber wrote the first parser.[^gruber]
Pandoc also allows inline notes.^[Defined right here.]

[^1]: See daringfireball.net/projects/markdown.
[^gruber]: First paragraph of the note.

    Indented paragraphs stay in the same note.

Euler's identity: $e^{i\pi} + 1 = 0$
GitHub-safe inline: $`\sqrt{x^2 + y^2}`$

$$
\sum_{i=1}^{n} i = \frac{n(n+1)}{2}
$$

It costs \$5, not \$10.
```

CODE EXPLAINED

- → `[^1]` — footnote marker; `[^1]:` — its definition
- → `^[text]` — inline footnote PANDOC
- → `$…$` — inline math
- → `$`…`$` — inline math that ignores Markdown GFM
- → `$$…$$` — display math (or a ```` ```math ```` fence)
- → `\$` — literal dollar sign

SUPPORT

FLAVOR	FOOTNOTES	MATH
CommonMark	✖	✖
GitHub	✔	✔
Pandoc	✔	✔
Obsidian	✔	✔
MkDocs	ext	ext
GitLab	✔	✔

LATEX ESSENTIALS

SOURCE	RESULT
`x^2 x_i`	Superscript, subscript
`x^{10}`	Group with {}
`\frac{a}{b}`	Fraction
`\sqrt{x} \sqrt[3]{x}`	Roots
`\sum_{i=1}^{n} \int_a^b`	Sum, integral
`\alpha \beta \pi \infty`	Greek, infinity
`\le \ge \ne \approx`	Comparisons

FOOTNOTE RULES

CASE	BEHAVIOR
`Undefined [^x]`	Prints literally
`Unused definition`	Dropped from output
`Same note twice`	One note, two backlinks
`Label order`	Numbered by first use
`Placement`	Definition can go anywhere

⚠ COMMON MISTAKES

- Unescaped `$` in prices — `$5 and $10` renders as math on GitHub
- Expecting `[^3]` to print "3" — notes renumber by order of use
- Underscores in inline math read as emphasis by some parsers — use `$`…`$`
- `$ x $` with inner spaces — not treated as math

✓ TIPS & BEST PRACTICES

- Use word labels (`[^source]`) so notes survive reordering
- Keep definitions right after the paragraph that uses them
- Put display math on its own lines with blank lines around it
- Preview math in the target tool; MathJax and KaTeX differ slightly

QUICK SUMMARY

- `[^id]` + `[^id]: text` — footnote
- `^[text]` — inline note (Pandoc)
- `$…$` — inline math
- `$$…$$` — display math
- `\$` — literal dollar

Front Matter & Attributes

METADATA & ATTRIBUTES

Front matter is a YAML block between `---` lines at the **very top** of a file. Site generators (Jekyll, Hugo, Astro, Docusaurus), Obsidian and Pandoc read it as page metadata; GitHub shows it as a table. **Attribute lists** like `{#id .class key=value}` attach HTML attributes to headings, images, links and blocks, and **fenced divs** (`:::`) wrap content in a styled container.

GOOD TO KNOW

- Front matter must start on line 1 — nothing before the first `---`
- YAML uses spaces, never tabs; quote values containing `:`
- Hugo also accepts TOML (`+++`) and JSON front matter

FRONT MATTER, ATTRIBUTES & DIVS

```
---
title: "Markdown Cheat Sheets"
date: 2026-10-04
tags: [markdown, reference]
---

## Install {#install .important}
![Logo](logo.png){width=40%} [Get it](app.zip){.button}

::: warning
Rendered as <div class="warning">.
:::

The HTML spec is maintained by the W3C.

*[HTML]: HyperText Markup Language
*[W3C]: World Wide Web Consortium
```

CODE EXPLAINED

- → `---` … `---` — YAML front matter (line 1)
- → `{#install .important}` — id + class on a heading
- → `{width=40%}` / `{.button}` — attributes on images, links PANDOC
- → `::: warning` — fenced div with class PANDOC
- → `*[HTML]:` — abbreviation; hover shows the expansion EXT

COMMON FRONT MATTER KEYS

KEY	USED FOR
`title`	Page / document title
`date`	Publish date (ISO)
`tags` / `categories`	Taxonomy
`draft: true`	Hide from build (Hugo)
`layout`	Template (Jekyll)
`sidebar_position`	Order (Docusaurus)

FRONT MATTER FORMATS

DELIMITER	FORMAT · USED BY
`--- … ---`	YAML · nearly everything
`+++ … +++`	TOML · Hugo
`{ … }`	JSON · Hugo
`% Title`	Pandoc title block

ATTRIBUTE SUPPORT

SYNTAX	WORKS IN
`## H {#id}`	Pandoc · Extra · MkDocs · kramdown
`{.class key=v}`	Pandoc · MkDocs · kramdown
`![a](i){width=…}`	Pandoc · MkDocs
`::: note`	Pandoc · Docusaurus
`*[ABBR]: text`	Extra · MkDocs (abbr)

⚠ COMMON MISTAKES

- A blank line or text before the first `---` — renders as a rule plus a heading
- `title: Markdown: A Guide` — unquoted colon breaks the YAML
- Tabs in YAML — parse error, often a silent build failure
- Using `{#id}` on GitHub — the braces show up in the heading

✓ TIPS & BEST PRACTICES

- Quote string values in YAML to be safe
- Use ISO dates: `2026-10-04`
- Keep front matter keys consistent across a site; document them
- Prefer fenced divs over raw HTML for styled blocks in Pandoc

QUICK SUMMARY

- `--- yaml ---` — front matter at line 1
- `+++ toml +++` — Hugo alternative
- `{#id .class}` — attributes (Pandoc, MkDocs)
- `::: name` — fenced div
- `*[ABBR]:` — abbreviation

Sheet 16

Raw HTML & Collapsible Sections

block vs inline HTML · blank-line rule · <details> · <summary> · markdown="1" · sanitized tags

HTML INSIDE MARKDOWN

Markdown passes **raw HTML** through. **Inline tags** (<kbd>, <sub>,
) work anywhere in text. An **HTML block** starts when a line begins with a block tag such as or <details>, and it ends at the next **blank line**. Markdown is not parsed inside the block until a blank line separates it. <details> + <summary> make **collapsible sections**. Hosted renderers like GitHub **sanitize** HTML and remove scripts and styles.

RULES

- Blank line after <summary> = Markdown works inside
- Indenting an HTML block 4+ spaces turns it into a code block
- <details open> starts expanded

COLLAPSIBLE SECTIONS & HTML

````
<details>
<summary>Click to expand</summary>

Markdown **works** here (blank lines around it).

```bash
npm run build
```

</details>

<details open>
<summary><b>Open by default</b></summary>

- lists work too

</details>

<div align="center">
  <img src="logo.png" width="120" alt="Logo">
</div>

Inline: H<sub>2</sub>O · <kbd>Esc</kbd> · line<br>break
````

CODE EXPLAINED

- → <details> — collapsible container
- → <summary> — the always-visible clickable line
- → Blank lines — let Markdown render inside the block
- → open — expanded on page load
- → — centering that survives GitHub
- → Inline tags — mix freely with text

WHERE HTML BLOCKS END

STARTS WITH	ENDS AT
<pre> <script> <style>	Matching closing tag
<!--	-->
<table> <details> …	Next blank line
Any other tag alone on a line	Next blank line
Inline tag mid-sentence	Not a block; inline HTML

GITHUB SANITIZER

KEPT	STRIPPED
<details> <summary>	<script> <style>
<p> <img> <a>	<iframe> <form>
<table> <kbd> <sub>	onclick= / on* handlers
<picture> <hr>	style= attributes
align width alt	<input> (except tasks)

⚠ COMMON MISTAKES

- No blank line after </summary> — the content shows as raw **text**
- Indenting HTML for neatness after a blank line — becomes a code block
- Expecting <style> or style= to work on GitHub
- Forgetting </details> — the rest of the page collapses into it

✓ TIPS & BEST PRACTICES

- Use <details> to hide long logs, FAQs and optional steps
- Prefer Markdown syntax first; HTML only where Markdown has no answer
- Keep HTML blocks flush left with blank lines around them
- Test in the target renderer; sanitizers differ by platform

QUICK SUMMARY

- <details><summary> — collapsible section
- blank lines — Markdown inside HTML
- <kbd> <sub>
 — inline HTML
- align="center" — GitHub-safe centering
- sanitizer — strips scripts and styles

Sheet 17

GitHub Markdown & READMEs

@mention · #123 · commit SHAs · :emoji: · badges · table of contents · relative links

MARKDOWN ON GITHUB

GitHub renders `.md` files, issues, pull requests, discussions and wikis with GFM plus extras: **@mentions**, **issue and PR references**, **commit links**, **emoji shortcodes**, alerts, footnotes, math and Mermaid. A **README.md** in the repo root, `.github/` or `docs/` is shown on the repo's front page, so it's the first thing visitors read.

GOOD TO KNOW

- References and mentions link only in comments, issues, PRs and rendered files — not inside code
- `Fixes #12` in a PR description closes the issue when the PR merges
- GitHub adds an auto-generated outline (TOC) menu to rendered files
- Relative links resolve against the branch being viewed

README SKELETON

```
# Project Name

[![npm](https://img.shields.io/npm/v/my-project)](https://npm.im/my-project)
[![License](https://img.shields.io/badge/license-MIT-blue)](LICENSE)

One sentence: what it does and who it's for.

## Install

```bash
npm install my-project
```

See [usage](docs/usage.md) and [contributing](CONTRIBUTING.md).

Fixes #42 — thanks @octocat :tada:
Related: owner/repo#7 and commit a5c3785.
```

CODE EXPLAINED

- → `[![alt](badge.svg)](url)` — status badge linking to its source
- → Short pitch first — visible without scrolling
- → `docs/usage.md` — relative link, works on every branch
- → `#42` / `owner/repo#7` — issue / PR links
- → `@octocat` — mention (notifies the user)
- → `a5c3785` — short commit SHA becomes a link

AUTO-LINKED REFERENCES

SYNTAX	LINKS TO
`@user @org/team`	User / team mention
`#123 GH-123`	Issue or PR in this repo
`owner/repo#123`	Issue or PR elsewhere
`a5c3785`	Commit (7+ hex chars)
`owner/repo@a5c3785`	Commit elsewhere
`Fixes #12`	Closes #12 on merge

REPO DOC FILES

FILE	PURPOSE
`README.md`	Front page of the repo
`CONTRIBUTING.md`	How to contribute
`SECURITY.md`	How to report vulns
`LICENSE`	License text
`.github/ISSUE_TEMPLATE/`	Issue forms / templates

EMOJI SHORTCODES

CODE	EMOJI
`:tada:`	🎉
`:rocket:`	🚀
`:warning:`	⚠️
`:white_check_mark:`	✅
`:x:`	❌
`:+1:`	👍

⚠ COMMON MISTAKES

- Linking repo files with absolute `…/blob/main/…` URLs — break on forks and other branches
- A wall-of-text README — readers bail before finding install steps
- Badges that don't link anywhere — wrap them in a link to the source
- Expecting `#12` to link inside a code span or block

✓ TIPS & BEST PRACTICES

- Lead with: what, why, install, quick example — in that order
- Use `<details>` for long optional sections
- Link with permalinks (press y on a file) so line refs don't drift
- Use `Closes` / `Fixes` / `Resolves #N` to auto-close issues

QUICK SUMMARY

- `README.md` — repo front page
- `@user` — mention
- `#123` / `owner/repo#123` — issue refs
- `Fixes #N` — auto-close on merge
- `:emoji:` — shortcodes

Obsidian & Chat-App Markdown

[[wikilink]] · ![[embed]] · ^block-id · %% %% · Slack mrkdwn · Discord · Jira wiki markup

OBSIDIAN & CHAT APPS

Obsidian stores notes as plain `.md` files and adds **wikilinks** (`[[Note]]`), **embeds** (`![[Note]]`), block references, tags, comments, highlights, callouts and YAML **properties**. **Chat apps** support only a subset of Markdown and often change the symbols: Slack uses single `*` for bold, Jira wiki markup uses `h1.` and `{code}`. Check the table before pasting Markdown into another tool.

OBSIDIAN NOTES

- Wikilinks resolve by note name, not path; renames update links
- `^block-id` at the end of a paragraph makes it linkable
- `#tag/sub` nests tags; a tag can't be all digits
- Settings → Files & links can switch new links to standard Markdown

AN OBSIDIAN NOTE

```
---
tags: [project, markdown]
aliases: [MD Sheets]
---
# Cheat Sheet Project
Related: [[Python Sheets]] · [[Prisma Sheets|Prisma]]
Section: [[Style Guide#Colors]]
Block: [[Meeting Notes#^decision1]]

![[Roadmap]]
![[cover.png|250]]

We ship on Friday. ^decision1

#status/in-progress  ==needs review==  %%private note%%

> [!todo] Next
> - [ ] Build sheets 08-22
```

CODE EXPLAINED

- → `[[Note]]` / `[[Note|text]]` — link / link with display text
- → `[[Note#Heading]]` — link to a heading
- → `[[Note#^id]]` — link to a block marked `^id`
- → `![[Note]]` — embed a note; `|250` sizes images
- → `#tag/sub` · `==hl==` · `%%hidden%%`
- → `> [!todo]` — callout (Sheet 07)

CHAT & TRACKER SYNTAX

FEATURE	SLACK	DISCORD	REDDIT	JIRA WIKI
Bold	`*b*`	`**b**`	`**b**`	`*b*`
Italic	`_i_`	`*i*`	`*i*`	`_i_`
Strike	`~s~`	`~~s~~`	`~~s~~`	`-s-`
Inline code	`` `c` ``	`` `c` ``	`` `c` ``	`{{c}}`
Code block	``` ```	```` ```lang ````	``` ```	`{code}…{code}`
Link	`<url|t> (API)`	`[t](url)`	`[t](url)`	`[t|url]`
Heading	—	`# H`	`# H`	`h1. H`
Quote	`> q`	`> q`	`> q`	`bq. q`

⚠ COMMON MISTAKES

- Pasting `**bold**` into Slack — shows the asterisks; Slack bold is `*b*`
- Using `[[wikilinks]]` in docs meant for GitHub — they render as literal brackets
- Writing Markdown in Jira Server/DC wiki fields — uses `h1.`, `{code}`, `[t|url]`
- Spaces in tags (`#in progress`) — tag stops at the space

✓ TIPS & BEST PRACTICES

- For portable notes, switch Obsidian to standard Markdown links
- Use block IDs for stable references to decisions
- Jira Cloud's editor accepts Markdown shortcuts as you type
- When unsure in chat apps, stick to `` `code` `` and plain URLs

QUICK SUMMARY

- `[[Note]]` — wikilink (Obsidian)
- `![[Note]]` — embed
- `^id` — block reference
- `*b*` — Slack / Jira bold
- `[t|url]` — Jira wiki link

Tooling: Pandoc, Linters & Site Generators

THE MARKDOWN TOOLCHAIN

Pandoc converts Markdown to and from almost any format: HTML, Word, PDF (through LaTeX), EPUB, slides. **Linters** like markdownlint catch structural problems, and **formatters** like Prettier rewrite files into one consistent style. **Static site generators** turn a folder of `.md` files into a website. **MDX** extends Markdown with JSX components for React-based doc sites.

GOOD TO KNOW

- Pandoc PDF output needs a LaTeX engine (or `--pdf-engine=typst` / `weasyprint`)
- markdownlint rules are numbered (`MD001`, `MD013` …); configure per project
- Prettier formats Markdown out of the box, including tables
- MDX is stricter than Markdown: unclosed tags and `<!-- -->` are errors

COMMON COMMANDS

```
# Pandoc — convert
pandoc README.md -o README.pdf                  # needs LaTeX
pandoc notes.md -o notes.docx --toc             # Word + table of contents
pandoc -s doc.md -o doc.html --metadata title="Doc"
pandoc in.docx -t gfm -o out.md                 # Word → GitHub Markdown
pandoc paper.md --citeproc --bibliography refs.bib -o paper.pdf

# Lint & format
npx markdownlint-cli2 "**/*.md"                 # report problems
npx markdownlint-cli2 --fix "**/*.md"           # auto-fix what it can
npx prettier --write "**/*.md" --prose-wrap preserve

# Static sites
pip install mkdocs-material && mkdocs serve     # http://127.0.0.1:8000
hugo new site blog && hugo server
npx create-docusaurus@latest my-site classic
```

CODE EXPLAINED

- → `-o file.ext` — output format guessed from the extension
- → `-s` — standalone file with `<head>`
- → `-t gfm` — write GitHub-Flavored Markdown
- → `--citeproc` — format citations from a `.bib` file
- → `--fix` — rewrite files to satisfy lint rules
- → `--prose-wrap` — `always` / `never` / `preserve`

KEY MARKDOWNLINT RULES

RULE	CHECKS
MD001	Heading levels go up one at a time
MD004	Consistent list markers
MD009	No trailing spaces
MD013	Line length (default 80)
MD022	Blank lines around headings
MD033	No inline HTML
MD041	First line is an H1

STATIC SITE GENERATORS

TOOL	NOTES
MkDocs	Python · `mkdocs.yml` · Material theme
Hugo	Go · very fast · TOML/YAML
Jekyll	Ruby · built into GitHub Pages
Docusaurus	React · MDX · versioned docs
VitePress	Vue · Vue components in Markdown
Astro	JS · content collections · MDX

CONFIG & MDX

SNIPPET	PURPOSE
`"MD013": false`	Rule off in `.markdownlint.json`
`markdownlint-disable`	Inline: `<!-- markdownlint-disable MD033 -->`
`import X from '…'`	MDX: import a component
`<X prop="a" />`	MDX: render it
`{/* note */}`	MDX comment

⚠ COMMON MISTAKES

- Running `pandoc … -o out.pdf` without LaTeX installed — fails with an engine error
- Turning on every lint rule — MD013 line length floods long-prose docs
- `--prose-wrap always` on a repo that uses one sentence per line — reflows everything
- Moving `.md` files to MDX without fixing HTML comments and bare `<`

✓ TIPS & BEST PRACTICES

- Commit a `.markdownlint.json` so everyone lints the same way
- Run markdownlint and Prettier in CI or a pre-commit hook
- Use `pandoc --list-output-formats` to see every target
- Pick MkDocs for plain docs, Docusaurus/Astro when you need components

QUICK SUMMARY

- `pandoc in -o out` — convert formats
- `markdownlint-cli2` — lint
- `prettier --write` — format
- `mkdocs serve` — preview a docs site
- `MDX` — Markdown + JSX

Best Practices & Pitfalls

one sentence per line · alt & link text · blank lines around blocks · snake_case emphasis · accidental lists · flavor mismatches

WRITING MARKDOWN WELL

Good Markdown reads cleanly as plain text **and** renders correctly everywhere. Use a predictable structure: one H1, headings that never skip levels, blank lines around every block, and fenced code with a language. Write for **accessibility**: meaningful alt text, descriptive link text and real table headers. Most rendering bugs come from a small set of **pitfalls**, listed below.

HOUSE STYLE

- One sentence per line → small, readable diffs
- `-` for bullets, `*` / `**` for emphasis, `---` for rules
- Lowercase, hyphenated file names: `getting-started.md`

A CLEAN DOCUMENT

````
# Deploy Guide

What this covers and who it's for, in one or two lines.

## Prerequisites

- Node.js 20 or later
- Access to the `prod` cluster

## Steps

1. Build the image.
2. Push it to the registry.
3. Run the deploy script:

   ```bash
   ./deploy.sh --env prod
   ```

See [Rollback](#rollback) if anything fails.

![Pipeline: build, test, then release](pipeline.png)
````

CODE EXPLAINED

→ One H1 title, then H2 sections — no skipped levels

→ Short intro sentence under the title

→ Blank line before and after every list, heading and fence

→ Code fence indented 3 spaces — stays inside step 3

→ Descriptive link text and alt text that states the point

ACCESSIBILITY CHECKLIST

ELEMENT	DO THIS
Images	Alt text that says what matters
Links	Text that makes sense alone
Headings	In order: H1 → H2 → H3
Tables	A real header row
Emoji	Don't replace words with them
Diagrams	Add a text summary

PITFALLS & FIXES

GOTCHA	FIX
`snake_case_name`	Wrap in backticks
`Map<T> in prose`	Backticks, or `<`
`1986. A year`	Escape: `1986\.`
`- 5 degrees`	Escape: `\- 5`
`Text\n---`	Blank line before `---`
`Trailing-space breaks`	Use `\` instead

BEFORE YOU PUBLISH

CHECK	HOW
`Renders right`	Preview in the target tool
`Links work`	markdown-link-check / lychee
`Style`	markdownlint
`Spelling`	cSpell / Vale
`Anchors`	Click every TOC link
`Flavor`	No extensions the tool lacks

⚠ COMMON MISTAKES

- Writing for one renderer and publishing to another — extensions show up as raw text
- Skipping blank lines "because it worked in preview" — stricter parsers break
- Images without alt text and links reading "here"
- Hard-wrapping lines at 80 characters — every edit rewraps and bloats diffs

✓ TIPS & BEST PRACTICES

- Add markdownlint + a link checker to CI
- Keep a short style guide in `CONTRIBUTING.md`
- Prefer plain Markdown; use HTML only where Markdown has no syntax
- Re-read the raw file — if it's hard to read, simplify

QUICK SUMMARY

- `one H1` — then ordered headings
- `blank lines` — around every block
- ```` ```lang ```` — always label code
- `alt + link text` — accessibility basics
- `lint + preview` — before publishing

Sheet 21

Quick Reference — Core Syntax

headings · emphasis · lists · links · images · code · blockquotes · tables

HEADINGS & TEXT

SYNTAX	RESULT	#
`# Title`	**Heading 1**	04
`## … ######`	Heading 2 – 6	04
`*italic*  _italic_`	*italic*	03
`**bold**`	**bold**	03
`***both***`	***both***	03
`~~strike~~`	~~strike~~ GFM	03
`line\`	Hard line break	02
`\*  \#  \_`	Literal character	02
`<!-- note -->`	Hidden comment	02

LISTS

SYNTAX	RESULT	#
`- item`	• bullet	05
`1. item`	1. numbered	05
`7. item`	List starts at 7	05
`  - child`	Nested under –	06
`   1. child`	Nested under 1.	06
`- [ ] task`	☐ task GFM	06
`- [x] done`	☑ done GFM	06
`1986\.`	Not a list	05
`Term ⏎ :  def`	Definition PANDOC	06

LINKS & IMAGES

SYNTAX	RESULT	#
`[text](url)`	Link	08
`[t](url "Tip")`	Link with tooltip	08
`[t][id]  [id]: url`	Reference link	08
`<https://x.com>`	Autolink	08
`[t](#heading)`	Jump to heading	04
`[t](docs/a.md)`	Relative file	08
`![alt](img.png)`	Image	09
`[![alt](img)](url)`	Linked image	09
`<img width="200">`	Sized image	09

CODE

SYNTAX	RESULT	#
`` `code` ``	code	10
`` `` a`b `` ``	a`b	10
```` ```lang ````	Fenced block + language	10
`~~~`	Fence, tilde style	10
```` ```` ````	Fence that wraps ```	10
`4-space indent`	Indented code block	10
```` ```diff ````	+/- line coloring	11
```` ```mermaid ````	Diagram	11

QUOTES & BREAKS

SYNTAX	RESULT	#
`> text`	Blockquote	07
`>> text`	Nested quote	07
`> [!NOTE]`	Alert GFM	07
`---  ***  ___`	Horizontal rule	07
`blank line`	New paragraph	02
` `	Line break	02
` `	Non-breaking space	02
`<kbd>Ctrl</kbd>`	Ctrl key	03

TABLES

SYNTAX	RESULT	#
`| a | b |`	Row of cells	12
`|---|---|`	Delimiter row (required)	12
`:---`	Left align	12
`:---:`	Center align	12
`---:`	Right align	12
`\\\|`	Pipe inside a cell	12
` `	Break inside a cell	12
`<table colspan>`	Merged cells	13

⚠ COMMON MISTAKES

- Missing blank lines between blocks — lists, headings and tables merge into paragraphs
- No space after `#`, `-` or `1.` — not parsed as syntax
- Wrong indentation in nested lists — items flatten or turn into code

✓ TIPS & BEST PRACTICES

- The **#** column points to the sheet with full details
- Badges: GFM PANDOC OBSIDIAN EXT mark non-CommonMark syntax
- When in doubt, preview in the tool you're publishing to

QUICK SUMMARY

- `blank lines` — separate every block
- `** * `` ` — bold, italic, code
- `[t](url)` / `![a](src)` — link / image
- ```` ```lang ```` — fenced code

Sheet 22

Quick Reference — Extended & Flavors

task lists · footnotes · math · Mermaid · alerts · front matter · <details> · flavor matrix

EXTENDED SYNTAX

SYNTAX	RESULT	#
`text[^1]`	Footnote marker	14
`[^1]: Note`	Footnote text	14
`$x^2$`	Inline math	14
`$$ … $$`	Display math	14
`==text==`	highlight OBSIDIAN	03
`H~2~O  x^2^`	Sub / sup PANDOC	03
`Term ⏎ :  def`	Definition list	06
```` ```mermaid ````	Diagram	11

CALLOUTS & HTML

SYNTAX	RESULT	#
`> [!TIP]`	GitHub alert	07
`> [!note]-`	Folded callout OBSIDIAN	07
`!!! note "T"`	Admonition (MkDocs)	07
`<details>`	Collapsible section	16
`<summary>`	Its visible title	16
`<sub> <sup>`	Sub / superscript	03
`<picture>`	Dark-mode image	09
`<div align="center">`	Centered (GitHub)	16

METADATA & PLATFORMS

SYNTAX	RESULT	#
`--- yaml ---`	Front matter	15
`{#id .class}`	Attributes	15
`::: warning`	Fenced div	15
`@user  #123`	Mention / issue GFM	17
`:tada:`	Emoji shortcode	17
`[[Note]]`	Wikilink OBSIDIAN	18
`![[Note]]`	Embed OBSIDIAN	18
`^block-id`	Block reference OBSIDIAN	18

FLAVOR SUPPORT MATRIX

FEATURE	COMMONMARK	GFM / GITHUB	PANDOC	OBSIDIAN	MKDOCS	SHEET
Tables	✖	✔	✔	✔	✔	12
Task lists	✖	✔	✔	✔	EXT	06
~~Strikethrough~~	✖	✔	✔	✔	EXT	03
Footnotes	✖	✔	✔	✔	EXT	14
Math $…$	✖	✔	✔	✔	EXT	14
Mermaid	✖	✔	EXT	✔	EXT	11
Alerts / callouts	✖	✔	✖	✔	EXT	07
Definition lists	✖	✖	✔	✖	EXT	06
{#id} attributes	✖	✖	✔	✖	EXT	15
YAML front matter	✖	✔	✔	✔	✔	15
==Highlight==	✖	✖	EXT	✔	EXT	03
[[Wikilinks]]	✖	✖	EXT	✔	EXT	18

⚠ COMMON MISTAKES

- Using an extension your renderer lacks — it shows up as literal `[^1]`, `==x==` or `[!NOTE]`
- Copying Obsidian notes to GitHub — wikilinks and embeds break
- Assuming GFM = GitHub — alerts, math, footnotes and Mermaid are GitHub features on top of the GFM spec

✓ TIPS & BEST PRACTICES

- ✔ supported · ✖ not supported · EXT needs an extension, plugin or filter
- MkDocs extensions mostly come from `pymdown-extensions`
- For portable docs, stick to the CommonMark + GFM columns

QUICK SUMMARY

- `[^1]` / `$…$` — footnotes, math
- `> [!NOTE]` — GitHub alert
- `---` yaml `---` — front matter
- `[[Note]]` — Obsidian link